

Ensemble: what to do next

Prepared 6 September 2026 | RG-GitUser/ensemble | branch fix/audit-remediation | PR #3

Where things stand

PR #3 is open and **CI is green for the first time** - all five steps passed (npm ci, typecheck, 61 tests, production build, dependency audit) on a clean Ubuntu install with Node 22. The branch carries 6 commits, 92 files, +5,726 / -600: the 57 audit findings, RTMPS stream keys, the social connection check, per-creator relay egress metering, sign-in with Google/Microsoft/Yahoo, and the fix that made CI actually run its tests.

Part 1 - Do these first

Ten minutes, and everything else builds on them.

- ☐ **Merge PR #3**
It is green and every later change stacks on it. Merging also puts ci.yml on main, which is what makes the push trigger start working for future commits.
- ☐ **Make the CI check required on main**
Settings -> Branches -> add a rule for main -> require the 'check' status. Right now GitHub reports mergeable_state=unstable, which means nothing enforces that CI must pass. Without this a future red build will not block a merge.
- ☐ **Reconcile or close origin/hardening**
6 commits, 32 files, overlapping scope with this branch, never merged. Leave it much longer and nobody will remember which of the two is authoritative.

Part 2 - Launch blockers that are not code

None of these can be written for you. The first is the longest pole - start it today.

- ☐ **Legal review by a qualified person**
Terms with governing law, dispute resolution and a refund policy (you bill monthly and deliberately do not prorate - that has to be somewhere a customer agreed to). A privacy page naming Stripe, Resend and DigitalOcean as subprocessors, with retention periods and data-subject rights. An acceptable-use policy with an abuse contact and a DMCA process.
- ☐ **Stripe: live mode and one full lifecycle**
Register the webhook against the six events billing.ts handles, then run it end to end: subscribe, upgrade, downgrade, fail a payment, cancel, delete the account - and confirm the card actually stops. That last step is the finding most likely to become a chargeback.
- ☐ **Configure Stripe Tax in the dashboard, THEN set the flag**
In that order. STRIPE_AUTOMATIC_TAX=1 before Tax is configured makes every checkout fail.

`STRIPE_AUTOMATIC_TAX=1 # only after the Stripe dashboard is configured`
- ☐ **Restore a backup onto a scratch droplet**
This has only ever been done on a laptop. It is the one step that turns a backup from a belief into a backup. Do it once before launch and once a quarter after.
- ☐ **Make one real post per social provider**
You have OAuth apps on prod now, but that path has never been proven against a live API. Treat the first post to each provider as a test, not as a launch. The new 'Check connection' button on Integrations verifies the token without publishing anything.

☐ **Register the SSO apps, if you want sign-in live**

Optional. Any provider missing either value is simply not rendered on the login page, so leaving these unset keeps email-and-password as the only way in. Redirect URIs are in .env.example.

```
GOOGLE_CLIENT_ID= / GOOGLE_CLIENT_SECRET=
MICROSOFT_CLIENT_ID= / MICROSOFT_CLIENT_SECRET=
YAHOO_CLIENT_ID= / YAHOO_CLIENT_SECRET=
```

One behaviour change before you deploy. With NODE_ENV=production and no STRIPE_SECRET_KEY, billing now **fails closed** and logs why. That is the fix for free-Enterprise-by-default. An install relying on the old permissive behaviour needs ENSEMBLE_BILLING_DISABLED=1 set on purpose.

Part 3 - Server security checklist

Run as root on the droplet unless a line says otherwise. Commands are from DEPLOY.md; each box is one thing to confirm rather than assume.

A. Accounts and firewall

☐ **Service account exists and is NOT in sudo**

Compromising the app must not be the same as owning the box. Administer as root or from your own sudo-capable login.

```
adduser ensemble
groups ensemble # must NOT list sudo
deluser ensemble sudo # only if it does
```

☐ **Firewall allows only SSH, HTTP and HTTPS**

```
ufw allow OpenSSH && ufw allow 80 && ufw allow 443 && ufw enable
ufw status verbose
```

B. Secrets and file permissions

☐ **The env file is not world-readable**

Every secret lives in it - Stripe, Resend, the live hook secret, the admin password. The default umask leaves it mode 644.

```
chmod 600 /srv/ensemble/.env
stat -c '%a %U:%G' /srv/ensemble/.env # expect: 600 ensemble:ensemble
```

☐ **ADMIN_PASSWORD was never copied from the example**

It used to ship set to a known value with the runbook telling you to copy the file, which defeated the random-password safety net four lines above it. It is a one-shot, permanent mistake against an admin address published in the repo.

```
grep -n '^ADMIN_PASSWORD' /srv/ensemble/.env # expect: no output
journalctl -u ensemble | grep -i 'admin password'
```

☐ **The relay has its OWN env file, not the app's**

EnvironmentFile loads a whole file, not one line. Pointing the relay at the app's .env put every platform secret into its environment and into every ffmpeg, curl and jq child it spawns.

```
printf 'LIVE_HOOK_SECRET=%s\n' 'the-value-you-generated' > /etc/ensemble-relay.env
chown root:ensemble /etc/ensemble-relay.env
chmod 640 /etc/ensemble-relay.env
```

C. Service hardening

☐ **systemd sandboxing is actually in effect**

These are set in `deploy/ensemble.service`. Confirm the running unit picked them up rather than trusting the file on disk.

```
systemctl show ensemble -p NoNewPrivileges -p ProtectSystem \
-p MemoryMax -p StartLimitBurst -p StartLimitIntervalSec
# expect NoNewPrivileges=yes, ProtectSystem=strict, a real MemoryMax
```

D. Backups

☐ **sqlite3 is installed**

Not optional: `backup.sh` and the whole documented restore shell out to it. Without it the nightly job died on its first command every night, silently, under `set -euo pipefail`.

```
apt-get install -y sqlite3
command -v sqlite3
```

☐ **The backups directory exists BEFORE the crontab line**

Load-bearing ordering. Cron's shell opens the redirect before the script runs, so on a fresh droplet the job never executed - and because the failure was in the redirect, there was no log to notice it in.

```
chmod +x /srv/ensemble/scripts/backup.sh
mkdir -p /srv/ensemble/backups
crontab -e -u ensemble
0 4 * * * /srv/ensemble/scripts/backup.sh >> /srv/ensemble/backups/backup.log 2>&1
```

☐ **rsync is configured as the SERVICE user, not root**

The cron job runs as `ensemble` and `rsync` reads its config from the running user's home. A remote configured as `root` is invisible to the job that needs it.

```
sudo -iu ensemble rsync config
grep -n '^BACKUP_REMOTE' /srv/ensemble/.env
```

☐ **A backup has been restored end to end, on a real box**

An untested backup is a belief, not a backup.

```
ls -la /srv/ensemble/backups/
tail -20 /srv/ensemble/backups/backup.log
```

Never back up the database with `cp`. It runs in WAL mode, so at any moment the most recent writes are in `app.db-wal` rather than `app.db`. Copying `app.db` alone gets you an almost empty database that still opens cleanly - the worst kind of bad backup. Use `sqlite3 .backup`, or take `app.db`, `app.db-wal` and `app.db-shm` together.

E. Live relay

☐ **RTMP is open; plain RTMP is closed to the world**

The ingest key travels in the URL path in cleartext, and creators stream from venue and hotel wifi. Only the local `ffmpeg` forwarders use 1935, over loopback, which the firewall does not touch.

```
ufw allow 1936/tcp
ufw deny 1935/tcp
chmod +x /srv/ensemble/deploy/live-push.sh
```

☐ **Set a relay egress allowance if 250 GB/month per creator is wrong for your droplet**

Optional. Forwarding one stream to three platforms is roughly 8 GB/hour, so one continuous streamer clears a 1 TB allowance in about five days. The quota is now enforced when a stream starts and never interrupts one already running.

```
LIVE_EGRESS_BYTES_PER_SITE=268435456000 # bytes; omit to use the plan default
```

F. Monitoring

☐ Caddy's log directory exists

```
mkdir -p /var/log/caddy && chown caddy:caddy /var/log/caddy
```

☐ An uptime check points at the health route and wakes a phone

1-5 minute interval. It touches the database, so it distinguishes 'up' from 'up and returning 500 to everyone' - which nothing outside the box could previously tell.

```
https://ensemble.it.com/api/health
```

☐ You know how to read the logs when it goes wrong

```
systemctl status ensemble # 'failed', not endlessly 'activating'
```

```
journalctl -u ensemble -n 50
```

```
tail -f /var/log/caddy/access.log
```

G. Verify from OUTSIDE the box

Run these from your laptop, not the droplet. Several of these controls only mean anything when tested from the internet.

☐ Health route answers

```
curl -s -o /dev/null -w '%{http_code}\n' https://ensemble.it.com/api/health # 200
```

☐ Security headers are present and http redirects to https

```
curl -sI https://ensemble.it.com | grep -iE 'strict-transport|x-content-type|referrer-policy'
```

```
curl -sI http://ensemble.it.com | head -1 # expect 301/308
```

☐ The on-demand-TLS ask endpoint is blocked at the edge

Blocking it is what stops outsiders enumerating your customers.

```
curl -s -o /dev/null -w '%{http_code}\n' \
'https://ensemble.it.com/api/domains/check?domain=example.com' # not 200
```

☐ The live hook routes refuse an unauthenticated caller

```
curl -s -o /dev/null -w '%{http_code}\n' \
'https://ensemble.it.com/api/live/targets?key=whatever' # expect 401
```

Part 4 - Still open in the code

Not blocking a launch, but worth knowing they are outstanding.

☐ Newsletter sending from the creator's own mailbox

The other half of the email work. Gmail caps at ~500 recipients/day (2,000 on Workspace), Outlook ~300, Yahoo similar, and all three treat list mail through a personal mailbox as a ToS matter. Proposed design: show the cap against their real subscriber count before they send, refuse rather than half-deliver, keep Resend for anyone above it.

☐ TikTok publishing

It sits in the UI registry as authType 'oauth' with no entry in OAUTH_PROVIDERS, so it connects handle-only and can never publish. Either wire the provider or drop it from the registry - right now it is a visible hole in a feature you are selling.

☐ Relay push retry

If one ffmpeg dies on a platform hiccup, that destination is gone for the rest of the stream and the creator is not told.

Deliberately not shipped untested: it is signal handling in a live path, and getting it wrong leaves stray pushes holding a frozen frame on someone's channel. Needs a real MediaMTX and a real stream to verify.

**Upload filenames are still guessable**

theme---. Renaming to random ids is a migration touching every stored config value, and the exposure is a draft's imagery rather than anything executable. Documented in the route and left deliberately.

Every command above is from DEPLOY.md as it stands on fix/audit-remediation. Items marked with a red box are the ones that cost real money or real trust if skipped.